

CENTRO UNIVERSITÁRIO UNIFECAP

ISABELLA DOMINGUES RAIO (RA: 156772)

**CONSTRUINDO UM PROJETO ÁGIL NO GITHUB: DA GESTÃO AO
CONTROLE DE QUALIDADE**

Sistema de Gerenciamento de Tarefas Operacionais para Startup de Logística

**TABOÃO DA SERRA - SP
2026**

ISABELLA DOMINGUES RAIO

CONSTRUINDO UM PROJETO ÁGIL NO GITHUB: DA GESTÃO AO CONTROLE DE QUALIDADE

Relatório acadêmico e técnico apresentado como requisito parcial para a obtenção de nota na disciplina de Engenharia de Software do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Centro Universitário UniFECAF.

**TABOÃO DA SERRA - SP
2026**

SUMÁRIO

1 INTRODUÇÃO	4
2 METODOLOGIA ÁGIL E GESTÃO VISUAL (KANBAN)	5
3 IMPORTÂNCIA DA MODELAGEM NA ENGENHARIA DE SOFTWARE	6
4 MODELAGEM DO SISTEMA ATRAVÉS DE DIAGRAMAS UML	7
4.1 DIAGRAMA DE CASOS DE USO	7
4.2 DIAGRAMA DE CLASSES	8
5 ENGENHARIA DE REQUISITOS E GESTÃO DE MUDANÇAS	9
6 CONTROLE DE QUALIDADE, TESTES UNITÁRIOS E INTEGRAÇÃO CONTÍNUA	10
7 COMPROVAÇÃO DE RESULTADOS (EVIDÊNCIAS DO GITHUB)	11
7.1 QUADRO KANBAN (GITHUB PROJECTS)	11
7.2 HISTÓRICO DE COMMITS NO REPOSITÓRIO	12
7.3 PIPELINE DE CI/CD (GITHUB ACTIONS)	13
8 CONCLUSÃO	14
9 REFERÊNCIAS	15

1 INTRODUÇÃO

No atual cenário de constante transformação digital, as organizações de desenvolvimento de software deparam-se com a necessidade de conciliar agilidade nas entregas e rigor no controle de qualidade. O presente relatório técnico e acadêmico descreve de forma circunstanciada a concepção, o planejamento e a implementação de um sistema web de gestão de tarefas operacionais, desenvolvido sob medida para atender às necessidades urgentes de uma startup do setor logístico. Este projeto foi executado sob a égide da TechFlow Solutions, uma empresa fictícia especializada no fornecimento de soluções digitais corporativas.

O domínio da logística exige dos sistemas operacionais características específicas de alta performance, fácil manutenção e confiabilidade. Diante dessas exigências, optou-se pelo desenvolvimento de uma API baseada em FastAPI, que implementa de maneira robusta os comportamentos previstos por um CRUD de controle de tarefas. Para viabilizar uma modelagem e uma execução coerentes com as melhores práticas da Engenharia de Software modernas, o projeto foi planejado metodologicamente desde sua base teórica, transitando pelo versionamento sistemático no GitHub até a consolidação de pipelines de integração contínua para a garantia da qualidade estrutural.

2 METODOLOGIA ÁGIL E GESTÃO VISUAL (KANBAN)

Uma das principais causas de falha em projetos ágeis reside na inadequada gestão do limite de trabalho em andamento (WIP) e no isolamento de informações de progresso. A falta de transparência sobre o estado real das atividades gera sobrecarga na equipe, atrasos e falhas graves de comunicação técnica e de negócios. Para mitigar ativamente esse risco na startup de logística, adotou-se o modelo ágil Kanban como framework de gestão visual de processos operacionais e de engenharia.

A operacionalização do Kanban foi estruturada diretamente na plataforma integrada do GitHub Projects, estabelecendo o fluxo visual unificado do ciclo de vida das tarefas por meio das seguintes colunas obrigatórias:

- **To Do (A Fazer):** Coluna destinada ao acúmulo de requisitos operacionais mapeados e prontos para serem trabalhados de acordo com a priorização estratégica.
- **In Progress (Em Progresso):** Concentração de atividades ativas da equipe de engenharia de software, onde o progresso de desenvolvimento acontece sob limites gerenciados de sobreposição.
- **Done (Concluído):** Estado definitivo aplicável apenas às atividades que cumpriram com todos os requisitos, passaram pelos critérios de aceitação e foram homologadas após a execução bem-sucedida de testes automatizados.

O uso do GitHub Projects como hub de gerenciamento visual garantiu transparência absoluta. A centralização de rascunhos de tarefas e problemas (Issues) vinculados ao código operacional elimina gargalos e assegura que os marcos de desenvolvimento sejam facilmente visualizados tanto pela equipe técnica de desenvolvimento de software quanto pelo cliente logístico.

3 IMPORTÂNCIA DA MODELAGEM NA ENGENHARIA DE SOFTWARE

A modelagem desempenha um papel fundamental na construção de sistemas estáveis de software, funcionando como o elo conceitual entre as intenções estratégicas e a lógica concreta de programação. Codificar de maneira impulsiva, sem uma reflexão metodológica prévia, costuma introduzir acoplamentos prejudiciais e problemas estruturais que inviabilizam a expansão saudável e o refatoramento do sistema no futuro.

Por intermédio da modelagem usando a Unified Modeling Language (UML), torna-se viável mitigar ativamente as inconsistências de escopo na fase inicial do projeto. A importância da aplicação desse padrão assenta-se nos seguintes pontos chave:

1. **Abstração Arquitetural:** Simplificação e isolamento das complexidades estruturais de dados antes que os desenvolvedores comecem a redigir código de produção.
2. **Comunicação Transparente:** Criação de diagramas universais que garantem um alinhamento mútuo entre gestores e técnicos, independentemente do nível de maturidade em programação.
3. **Redução da Dívida Técnica:** Facilitação da tomada de decisão estrutural, evitando modificações complexas de código que seriam imensamente dispendiosas após a sua consolidação e implantação operacional.

4 MODELAGEM DO SISTEMA ATRAVÉS DE DIAGRAMAS UML

Para orientar o desenvolvimento técnico da API de gestão de tarefas operacionais, foram projetados dois diagramas de alta importância para o escopo logístico da TechFlow Solutions.

4.1 DIAGRAMA DE CASOS DE USO

O Diagrama de Casos de Uso estabelece os limites do sistema e ilustra graficamente o escopo de atuação do usuário final do software. Nele, o ator "Operador Logístico" interage com a fronteira do sistema web por meio de quatro ações fundamentais do CRUD: Criar Tarefa de Entrega, Listar Tarefas Operacionais, Atualizar Status e Excluir Tarefas do Fluxo.

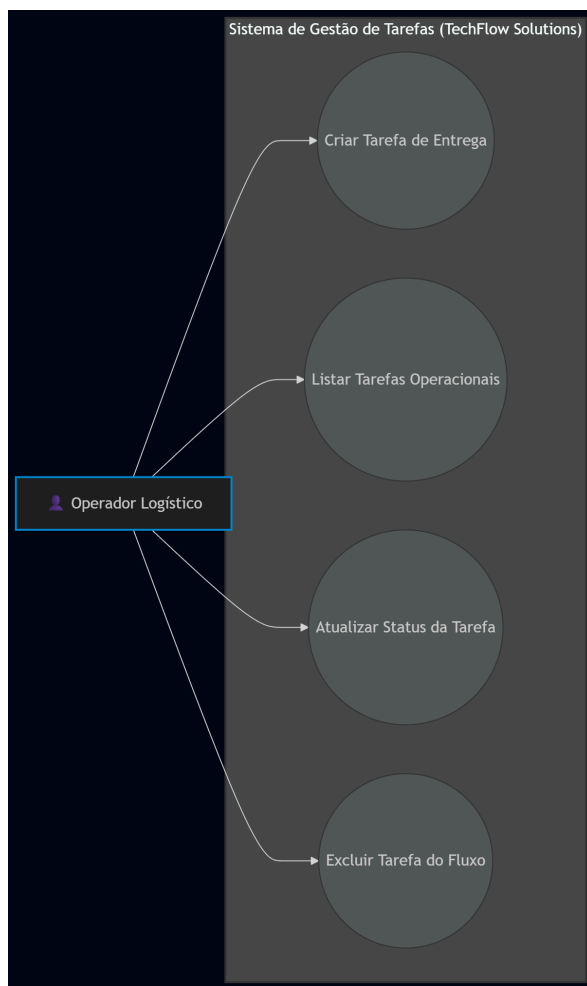


Figura 1: Diagrama de Casos de Uso - Escopo de Operações e Fronteira do Sistema. Fonte: Elaborado pelo autor (2026).

4.2 DIAGRAMA DE CLASSES

O Diagrama de Classes aborda a infraestrutura de dados estática do projeto. Ele mapeia as entidades e os tipos primitivos de variáveis responsáveis pela integridade das transações de backend. A classe principal "Tarefa" concentra os atributos essenciais para a rastreabilidade do processo logístico, contendo: id: int (identificador chave), titulo: str, descricao: str, status: str e prioridade: str (atributo incorporado como modificação).

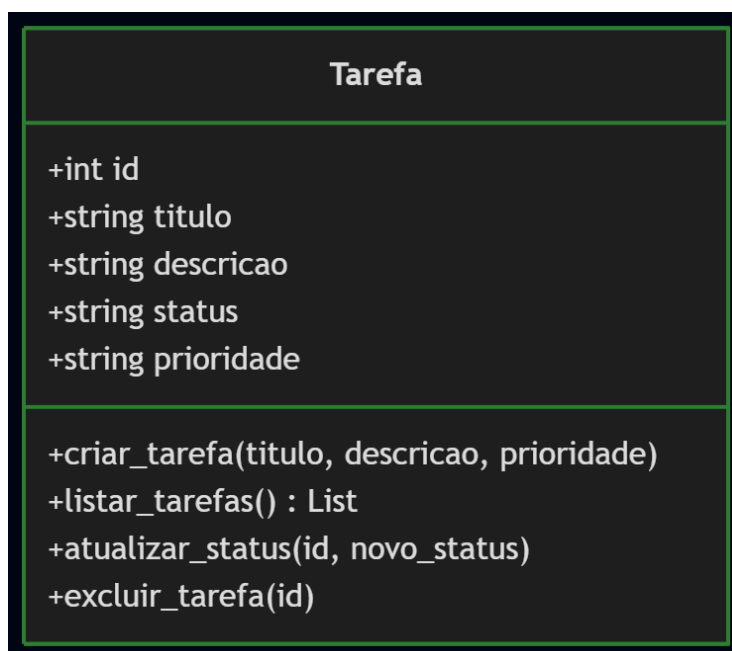


Figura 2: Diagrama de Classes - Estrutura Estática e Definição de Tipagem. Fonte: Elaborado pelo autor (2026).

5 ENGENHARIA DE REQUISITOS E GESTÃO DE MUDANÇAS

Em projetos ágeis reais, o escopo não é estático; ele evolui conforme o feedback contínuo do cliente e a evolução do entendimento de mercado. Essa flexibilidade é crucial na Engenharia de Software ágil. No entanto, o desafio reside em gerenciar as mudanças sem desestruturar o cronograma ou comprometer a integridade funcional do código preexistente.

Durante as iterações do projeto, a startup de logística constatou uma limitação crítica: o modelo inicial não permitia a identificação ou ordenação imediata de tarefas de urgência máxima (ex.: carregar suprimentos médicos ou resolver gargalos em frotas bloqueadas). Diante dessa lacuna operacional, solicitou-se formalmente a introdução de uma **alteração de escopo**.

Para absorver essa mudança de maneira estruturada, realizou-se o seguinte processo de gestão ágil de mudanças:

- **Atualização do Quadro Kanban:** Inclusão de um card estratégico focado na implementação e acompanhamento do novo atributo no GitHub Projects.
- **Alteração Controlada no Modelo de Dados:** Acoplagem do campo prioridade (com valor padrão determinado como "Média") no arquivo modelos.py por meio de tipagem Pydantic.
- **Garantia de Retrocompatibilidade:** Salvaguarda das operações básicas do CRUD de forma a garantir que as tarefas legadas criadas sem o atributo específico de prioridade continuassem sendo resolvidas perfeitamente.

6 CONTROLE DE QUALIDADE, TESTES UNITÁRIOS E INTEGRAÇÃO CONTÍNUA

Em ambientes de alta velocidade de entrega, a execução mecânica de testes manuais a cada modificação incremental no código é economicamente inviável e tecnicamente falha. A Engenharia de Software moderna delega a responsabilidade da estabilidade funcional e estrutural à automação, minimizando a ocorrência de erros conhecidos como regressões de software.

O controle de qualidade deste projeto foi estabelecido com a integração do framework de testes **Pytest** ao **FastAPI TestClient**. Foram programados testes automatizados focados nas seguintes regras de negócio logísticas:

- **Validação de Entrada Positiva:** Garantia de que novas tarefas com títulos e prioridades (Alta, Média ou Baixa) são criadas com sucesso (HTTP status 201).
- **Validação de Entrada Negativa:** Rejeição e tratamento de payloads contendo valores vazios ou nulos na propriedade estrutural de título (HTTP status 422).
- **Rastreabilidade de Registros Inexistentes:** Certificação de que buscas direcionadas a identificadores (ID) ausentes no banco de dados temporário retornam o tratamento de exceção adequado (HTTP status 404).

A segurança operacional foi consolidada pela infraestrutura de Integração Contínua (CI) provida pelo **GitHub Actions**. O workflow configurado no arquivo ci.yml orquestra na nuvem a criação de ambientes virtuais isolados a cada commit (Push) ou proposta de mesclagem de código (Pull Request). O código em produção somente é classificado como seguro se todos os testes escritos apresentarem validação de conformidade absoluta.

7 COMPROVAÇÃO DE RESULTADOS (EVIDÊNCIAS DO GITHUB)

Esta seção destina-se ao registro visual comprobatório da correta orquestração das ferramentas do GitHub, atestando a conclusão técnica e ágil exigida.

7.1 QUADRO KANBAN (GITHUB PROJECTS)

O quadro visual contendo os dez cards estratégicos distribuídos entre as colunas essenciais do Kanban planejado para o fluxo da startup de logística:

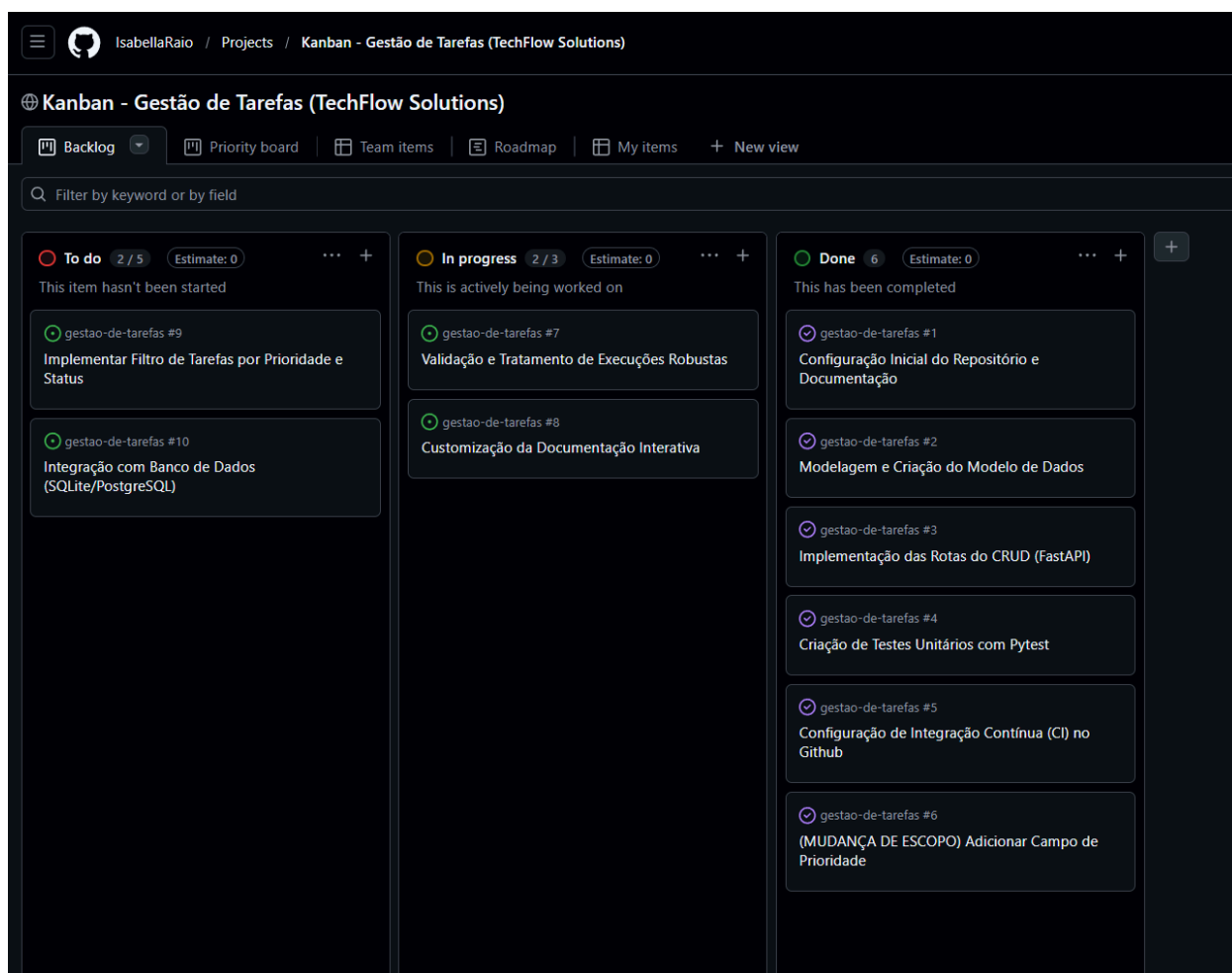


Figura 3: Quadro Kanban do Projeto Organizador no GitHub. Fonte: Elaborado pelo autor (2026).

7.2 HISTÓRICO DE COMMITS NO REPOSITÓRIO

O histórico evidenciando o desenvolvimento incremental através da consolidação dos dez commits com uso de mensagens semânticas (`feat`, `test`, `ci`, `docs`):

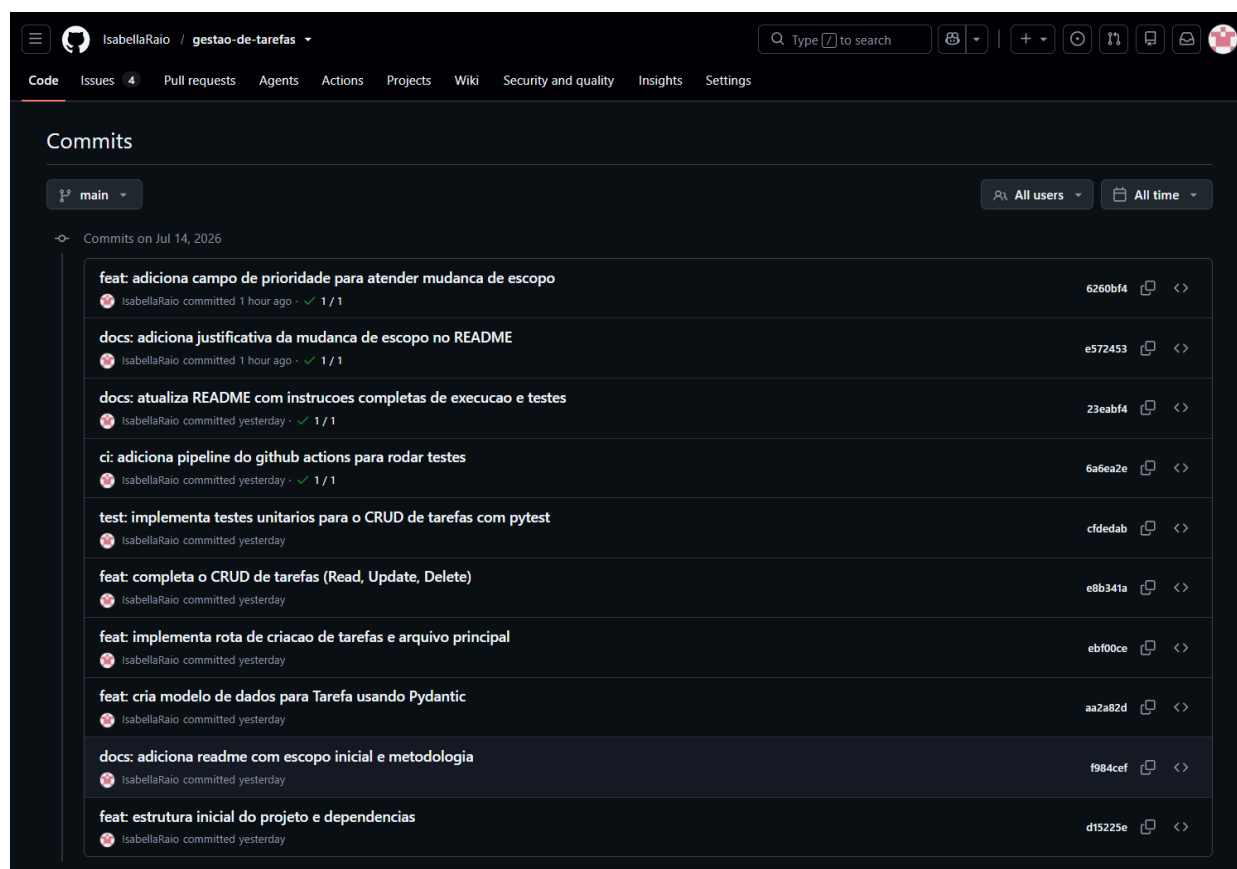


Figura 4: Histórico Sequencial de Commits Semânticos no Repositório. Fonte: Elaborado pelo autor (2026).

7.3 PIPELINE DE CI/CD (GITHUB ACTIONS)

A execução bem-sucedida do pipeline de integração contínua (CI), sinalizada com o ícone de sucesso verde na seção correspondente do GitHub Actions:

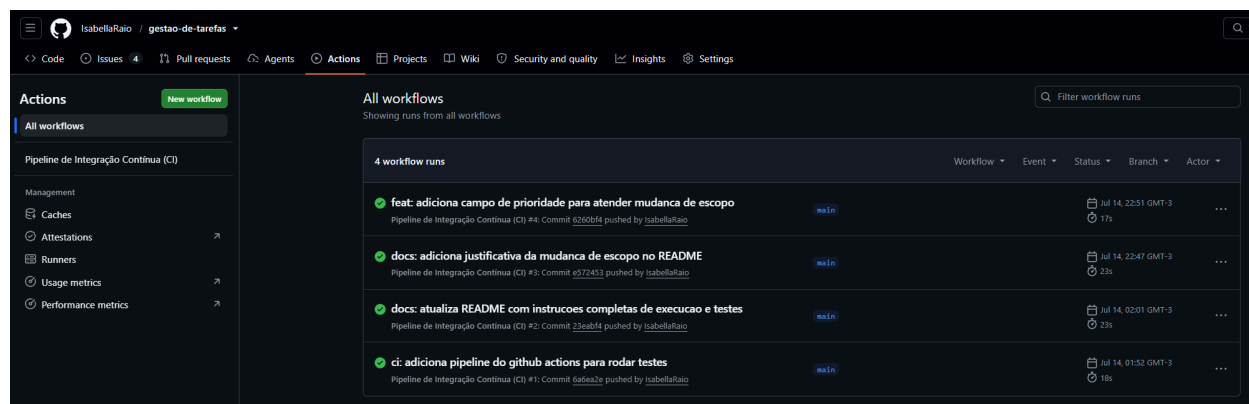


Figura 5: Execução Verde do Pipeline de Testes Automatizados no GitHub Actions. Fonte: Elaborado pelo autor (2026).

8 CONCLUSÃO

O desenvolvimento deste projeto técnico e acadêmico proporcionou uma imersão profunda na realidade prática das metodologias ágeis e do ciclo de vida de desenvolvimento de software modernas. A experiência de conceber, versionar, testar e documentar de ponta a ponta uma API utilizando FastAPI demonstrou que o valor de um produto de software de qualidade reside no equilíbrio dinâmico entre ferramentas automatizadas, organização de equipes e processos ágeis bem estruturados.

A integração e o gerenciamento conjunto de código-fonte e gestão visual de tarefas através do Kanban e GitHub Projects mostraram-se eficazes na eliminação de ambiguidades, na prevenção de desvios de escopo e no combate aos desperdícios de retrabalho na startup de logística fictícia. Do mesmo modo, a modelagem via diagramas UML antecipou o escopo arquitetural, facilitando modificações de negócio que puderam ser integradas de forma indolor graças à presença indispensável de testes unitários automatizados.

Portanto, este projeto comprova na prática as teorias consolidadas na disciplina de Engenharia de Software do Centro Universitário UniFECAF. Ao final desta jornada, o repositório produzido serve como evidência prática de uma arquitetura limpa, documentação exemplar e compromisso incondicional com padrões profissionais de excelência.

9 REFERÊNCIAS

ATLASSIAN BLOG. **Como Usar o Kanban no GitHub para Melhorar a Produtividade**. Disponível no repositório de recursos da Atlassian. Acesso em: 2026.

GITHUB. **Introdução ao GitHub Actions**. Disponível na Documentação Oficial do GitHub Docs (GitHub - Actions). Acesso em: 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software**: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.